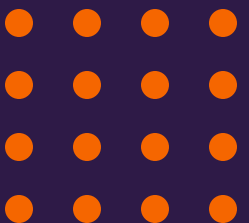
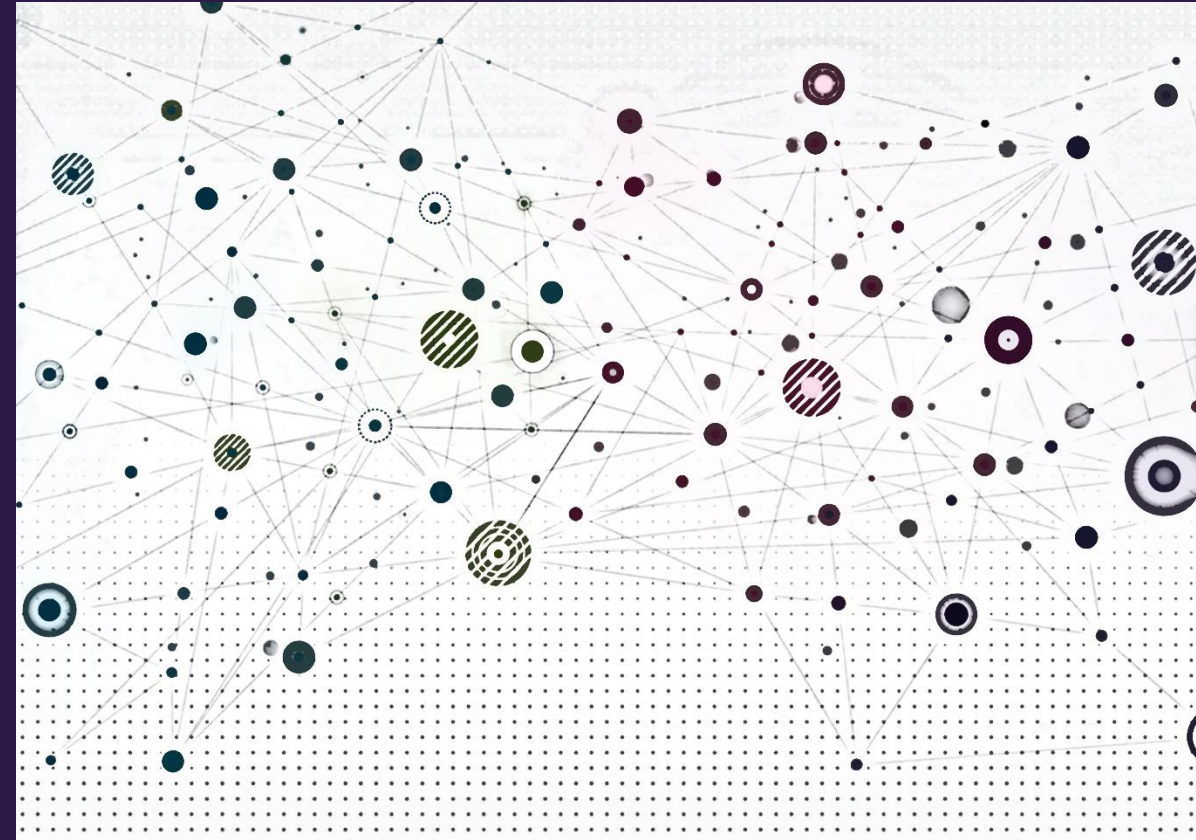


Introduction

Aaron J. Masino, PhD

Associate Professor of Computer Science





Overview

- Course logistics
- Motivations for graphs
- Rationale for specialized methods
- Terminology
- Machine learning tasks on graphs



Course logistics



Syllabus Review

Course Project Review

READ ME!

Complete details on course project requirements are on Canvas:

[ML4G-8810-Course-Project-Details.pdf](#)



Course project

- Goals
 - Gain experience with implementing and evaluating graph machine learning models on a project that is meaningful to you
 - Create a lasting resource for your technical profile - lab PIs and hiring managers love these
- Three types of projects
- Real-world application of graph neural networks (GNNs)
 1. In-depth web-based tutorial on PyTorch Geometric (PyG) functionality
 2. Implementation of new research in PyG
- (Opt In) If you write a blog post for your project, I will add it to the course blog.



1. Real-world application research with GNNs

- Goal: identify a use case of interest to you and demonstrate how GNNs can be applied
- Portfolio Output
 - Blog post or final report, code files
- Example applications
 - Predicting candidate drugs
 - Product recommendation
 - Internet traffic anomaly detection



2. PyG Tutorial

- Goal: develop a web-based tutorial that provides an in-depth explanation of some existing PyG functionality
- Portfolio Output
 - Blog post (**required for this project type**)
 - code files
- Example tutorial topics (PyG docs include minimal tutorials for these, but they assume a lot of background knowledge)
 - [PyG's GraphGym](#)
 - [Multi-GPU Training](#)



3. Research method implementation

- Goal: implement a method from a recent (2025, preferably) research paper on machine learning for graphs
- Portfolio Output
 - Final report or blog post
 - github repository & demonstration code files
 - (Extra credit) pull request to PyG [contrib](#)
- Portfolio requirements
 - Implementation should include comprehensive testing and documentation
 - Build on existing PyG and PyTorch code where possible
 - **Warning:** This project type is only recommended if you are already familiar with PyG and PyTorch and if team of three members is committed to the project!

Course Project Review

READ ME!

Complete details on course project requirements are on Canvas:

[ML4G-8810-Course-Project-Details.pdf](#)



Overall course surveys

- This is a new course and there will be room for improvement.
- Surveys about the course will be released during the semester
 - Your feedback is incredibly valuable
 - I will keep them brief (2-3 questions)
 - Submissions are anonymous

Supplemental course materials

- Throughout the slides, there are links to supplementary materials
- Materials include interactive web pages and Jupyter notebooks
- Materials further illustrate concepts
- Indicated by URL in a blue box:

<https://ml4g.masinolab.com/index.html>

Interactive Resources

Machine Learning for Graphs

NETWORK METRICS

- Constant Edge Density
- Edge Density with Constant Degree
- Walks and Paths in Networks
- Number of Walks in a Network
- Network Clustering Coefficient

METRIC ALGORITHMS AND COMPLEXITY

- Network Node Degree Complexity

[Notebook](#)

Machine Learning for Graphs

Supplemental Materials for Machine Learning for Graphs

Welcome to the Interactive Resource Hub

This website provides interactive demonstrations, visualizations, and you develop a deeper understanding of key concepts in network and graph machine learning.

Navigate through the topics using the sidebar to explore interactive resources.

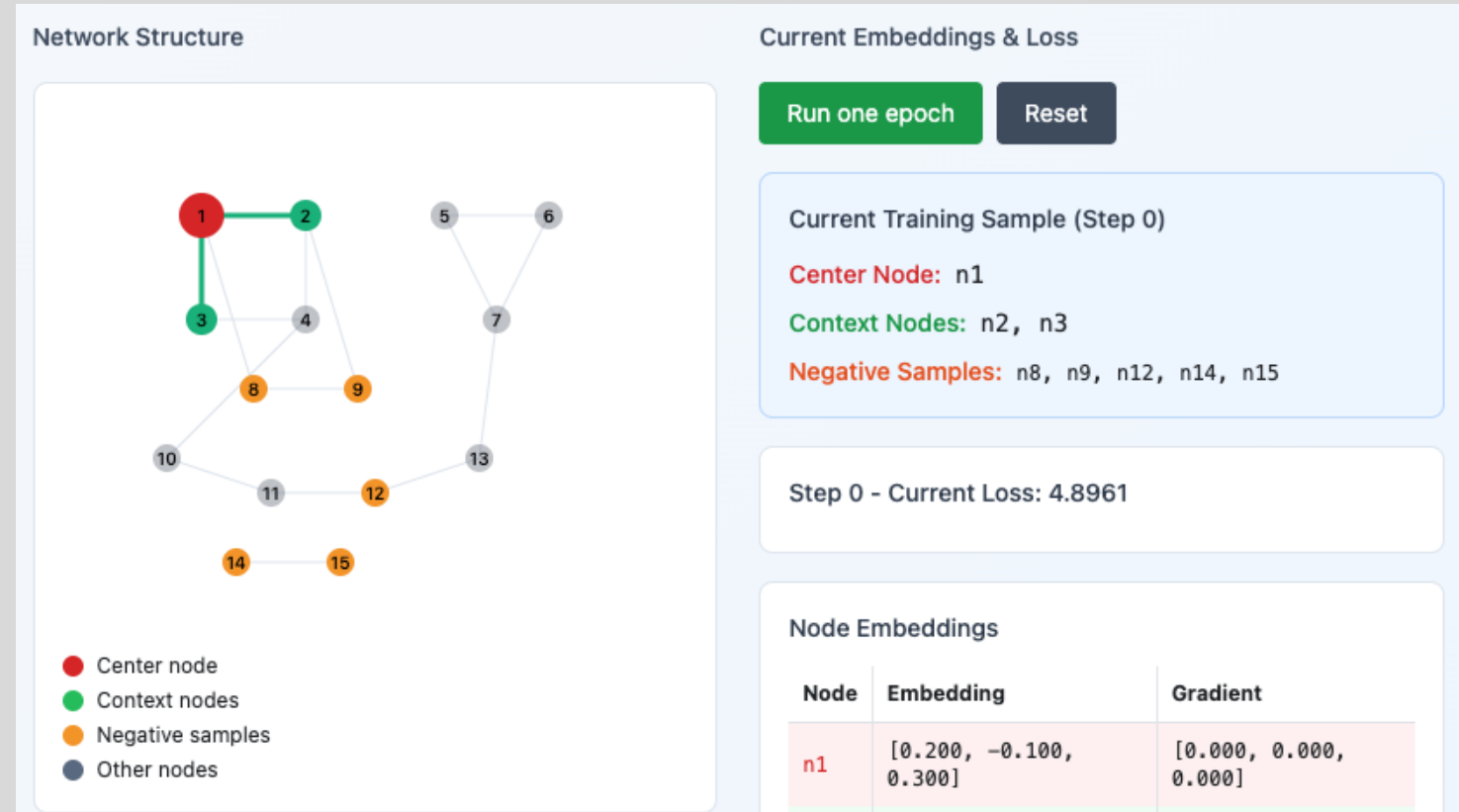
Note

These resources are supplementary to the main course content and can be accessed at any time during the course.



Supplemental materials an experiment with AI

- The supplementary materials are all created with generative AI
- Student perception of the quality and value of the materials is of interests (to me)
- Surveys related to these materials will be released during the semester
 - They will be brief
 - Anonymous





Motivations for graphs





A language for entities and relationships

- Graphs provide a general language for describing and analyzing entities and their relationships
- Many complex systems and phenomena of interest have a natural graph representation

Technological Networks as Graphs

A. Computer Networks

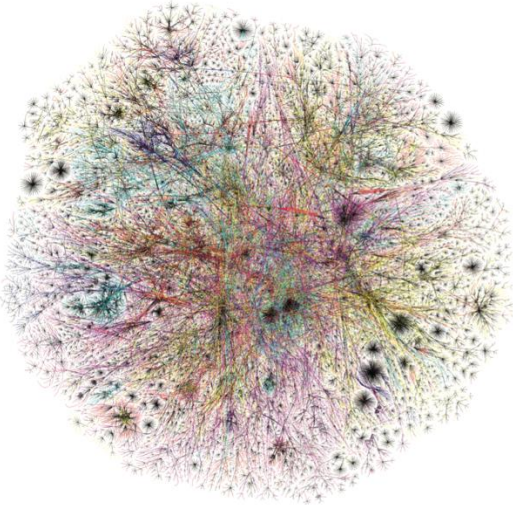


Image credit: Newman 2018

B. Gas Pipelines

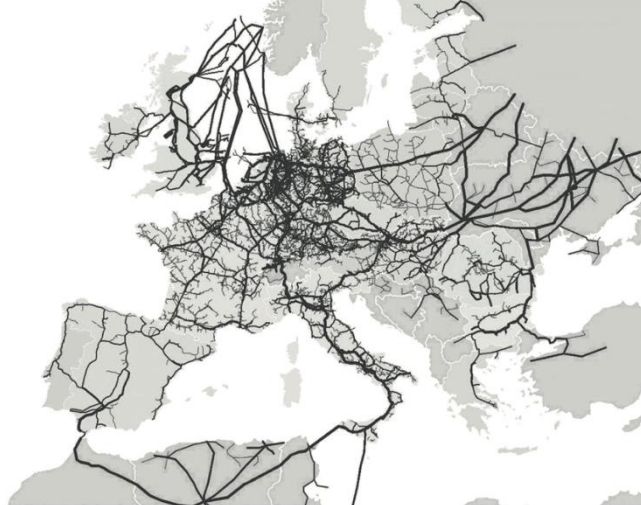


Image credit: Newman 2018

C. Transportation



Image credit: [Go!NewYork](#)

D. Communication Networks

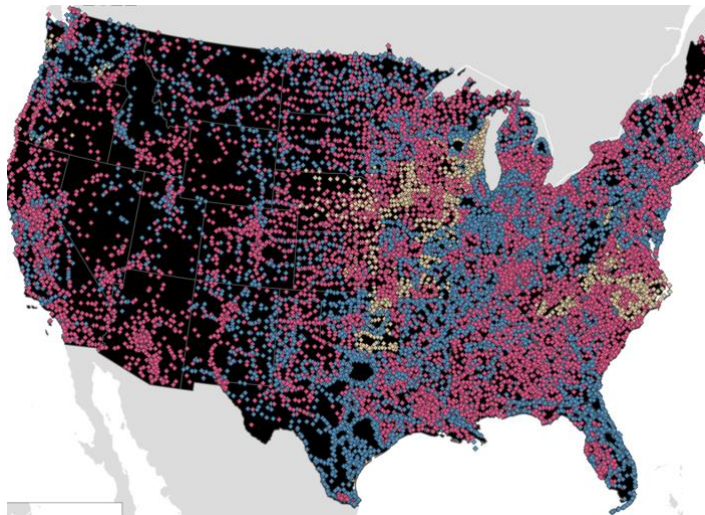


Image credit: [soundingmaps](#)

E. Power Grid

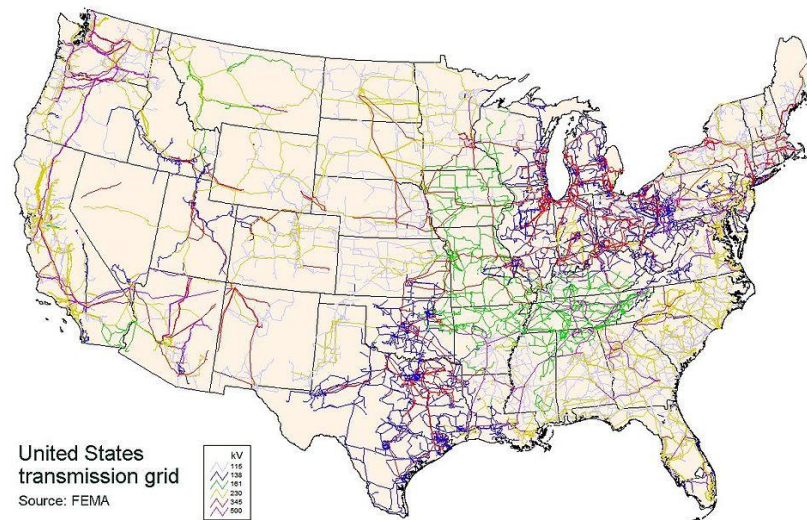


Image credit: [wikimedia commons](#)

F. Water Distribution

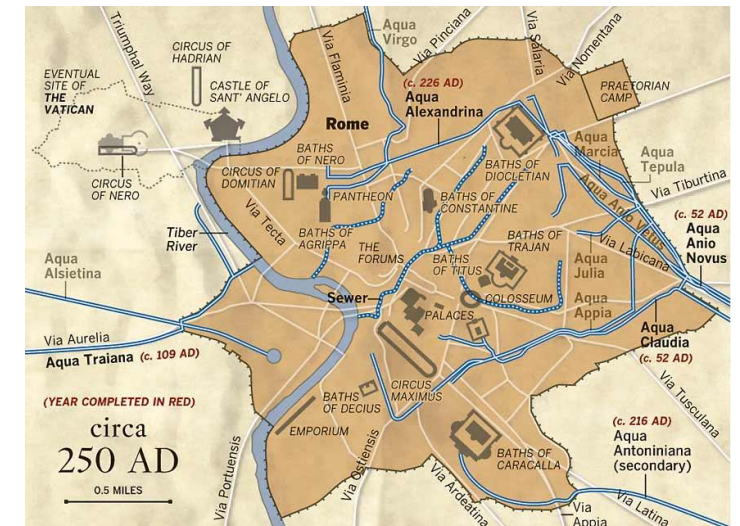


Image credit: [Los Angeles Times](#)

Information Networks as Graphs

A. World Wide Web

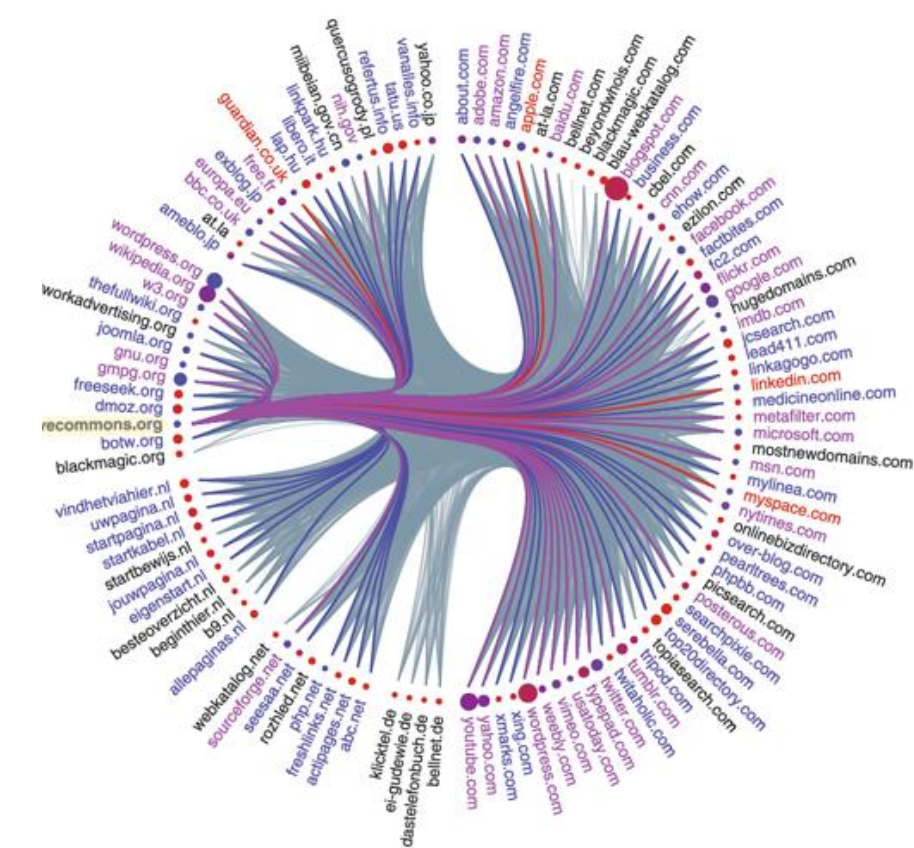
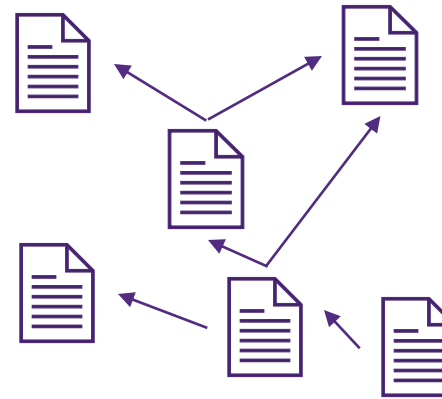


Image credit: [American Scientist](#)

B. Citation Network



C. Databases

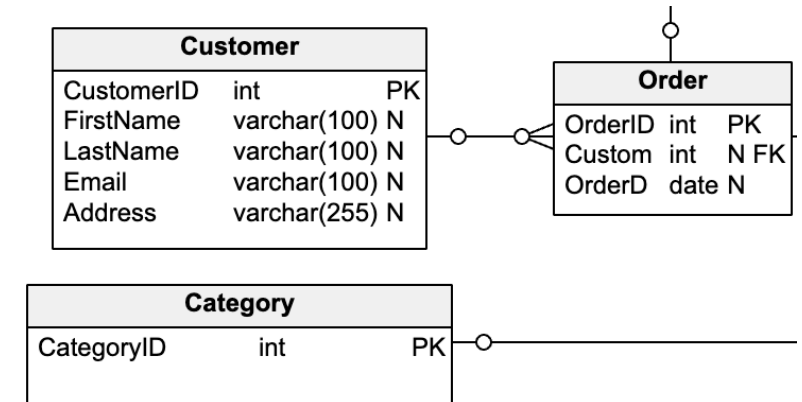
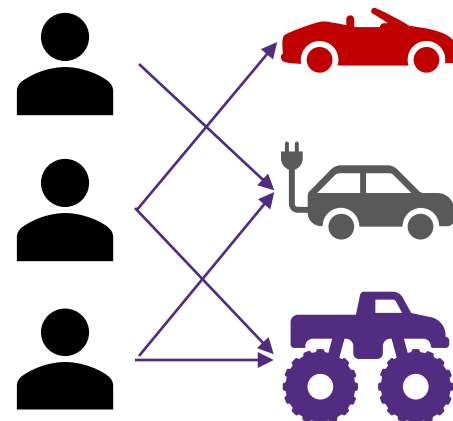
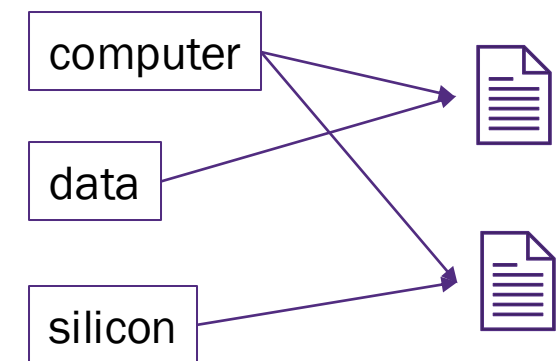


Image credit: [vertabelo](#)

D. Recommender Network



E. Keyword Index



Social Networks as Graphs

A. Marriage Network

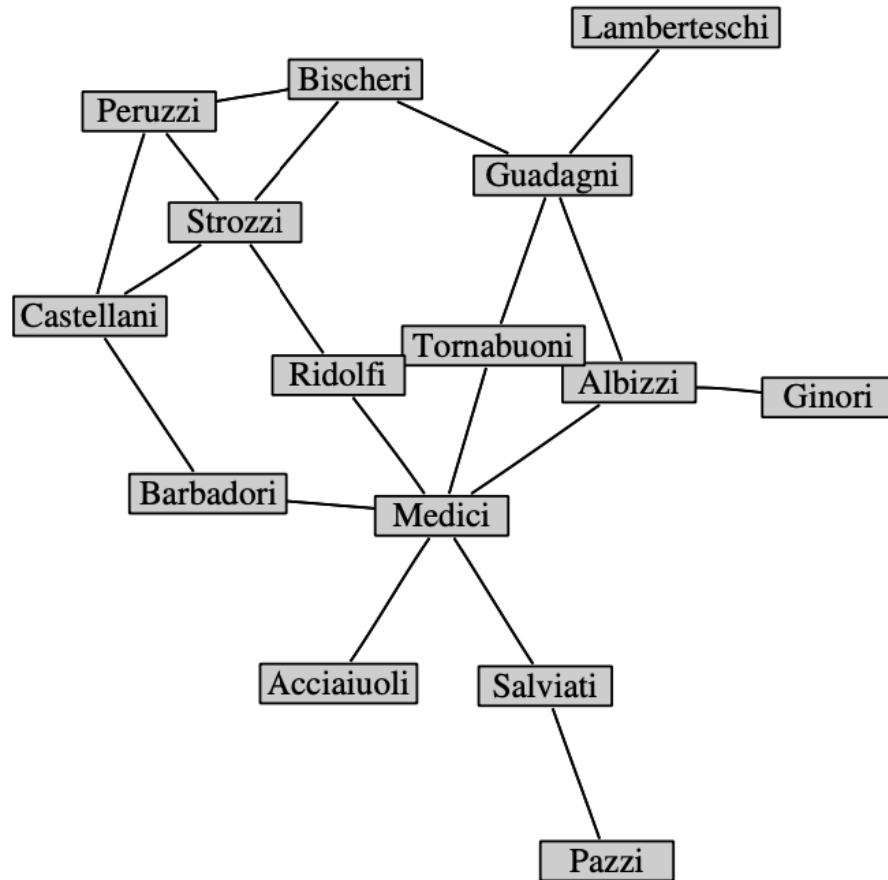


Image credit: Newman 2018

B. Social Media Networks



Biological Networks as Graphs

A. Metabolic Pathways

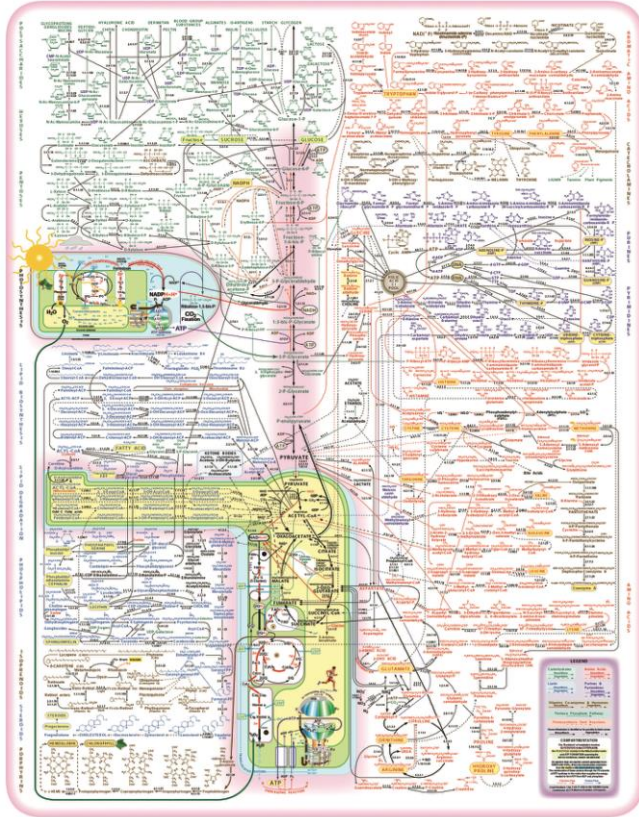


Image credit: Newman 2018

A. Protein Interactions

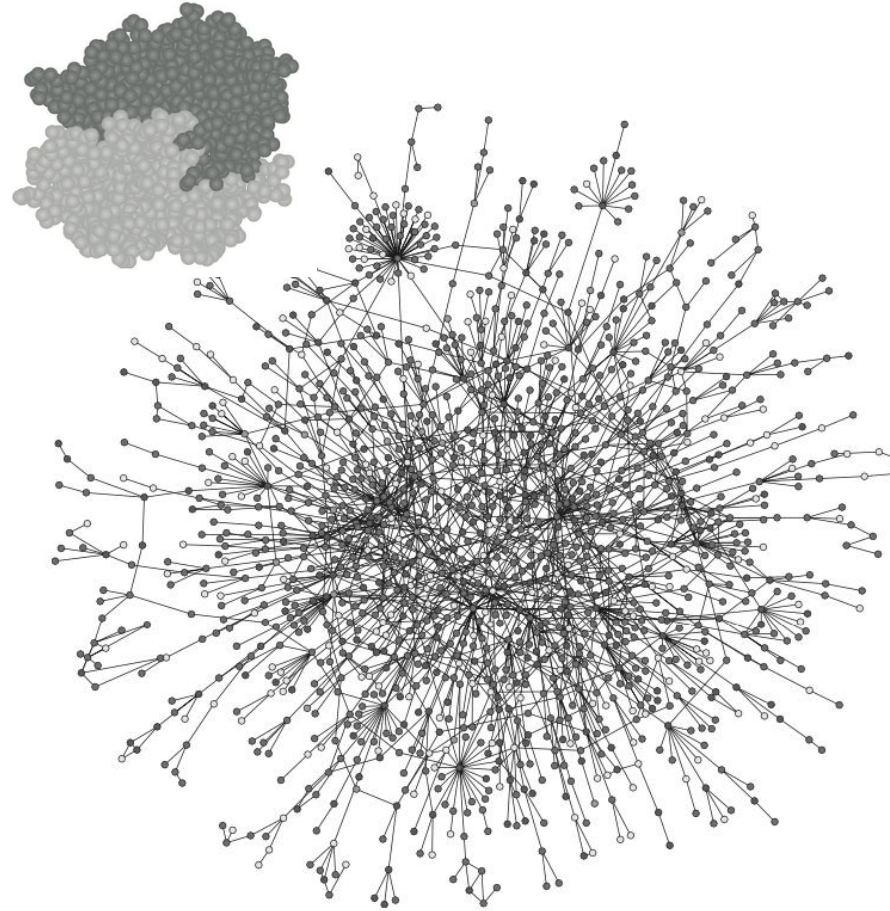


Image credit: Newman 2018

A. Neurons

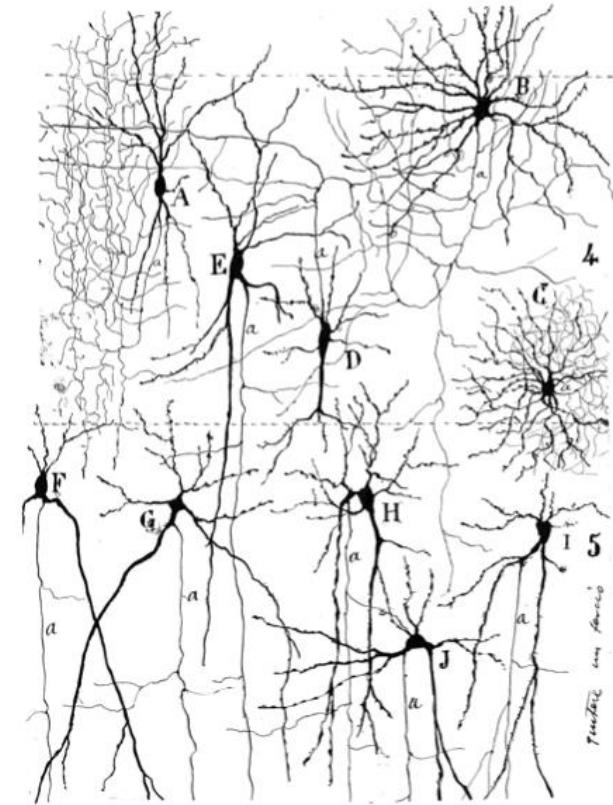


Image credit: Newman 2018

Other Graphs

A. Software execution

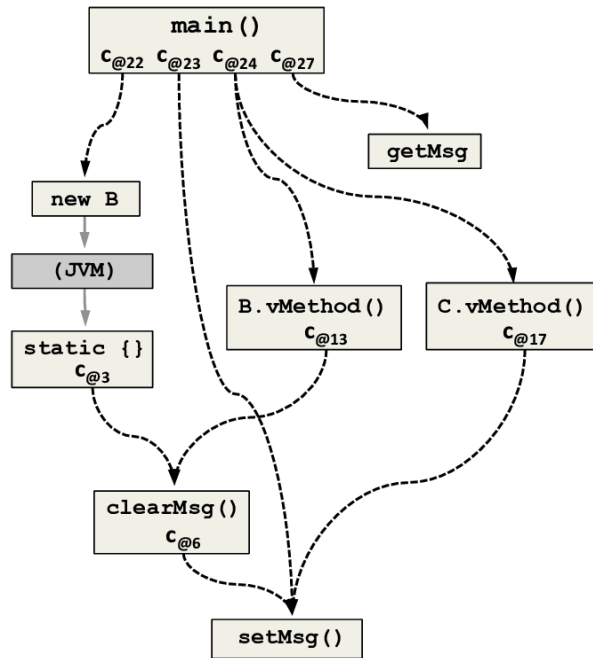


Image credit: [ResearchGate](#)

C. Molecules

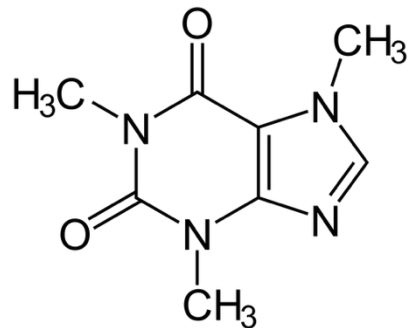


Image credit: [Chemistry StackExchange](#)

B. Computation

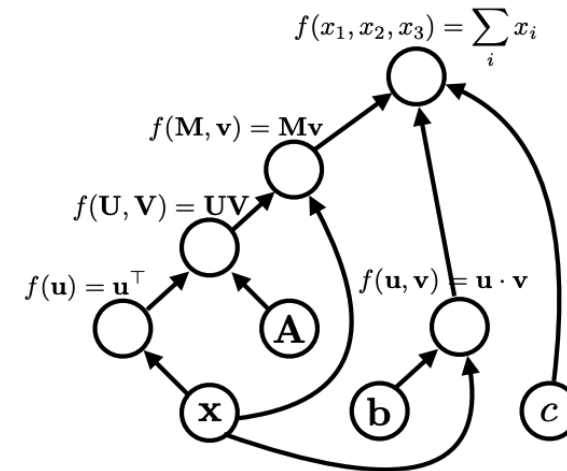


Image credit: [Cornell University](#)

D. Knowledge graphs

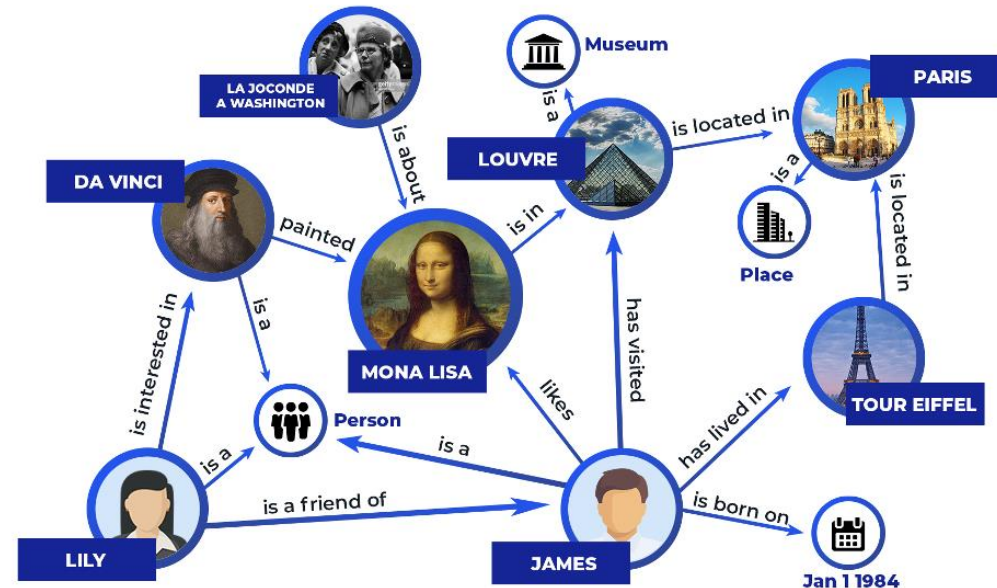


Image credit: [Zilliz](#)



Rationale for specialized methods





Standard machine learning assumptions are violated

Independence

- Standard machine learning assumes samples are statistically independent
- Nodes in a graph are not usually independent. Knowing something about a node typically provides information about its neighbor nodes

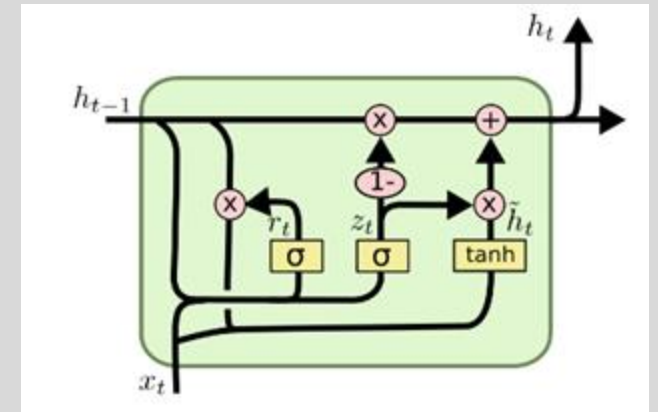
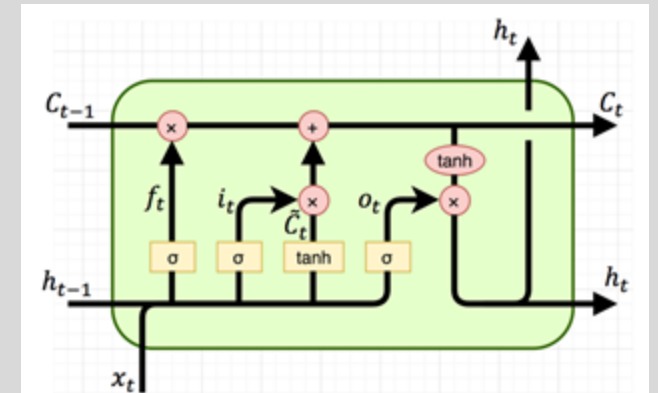
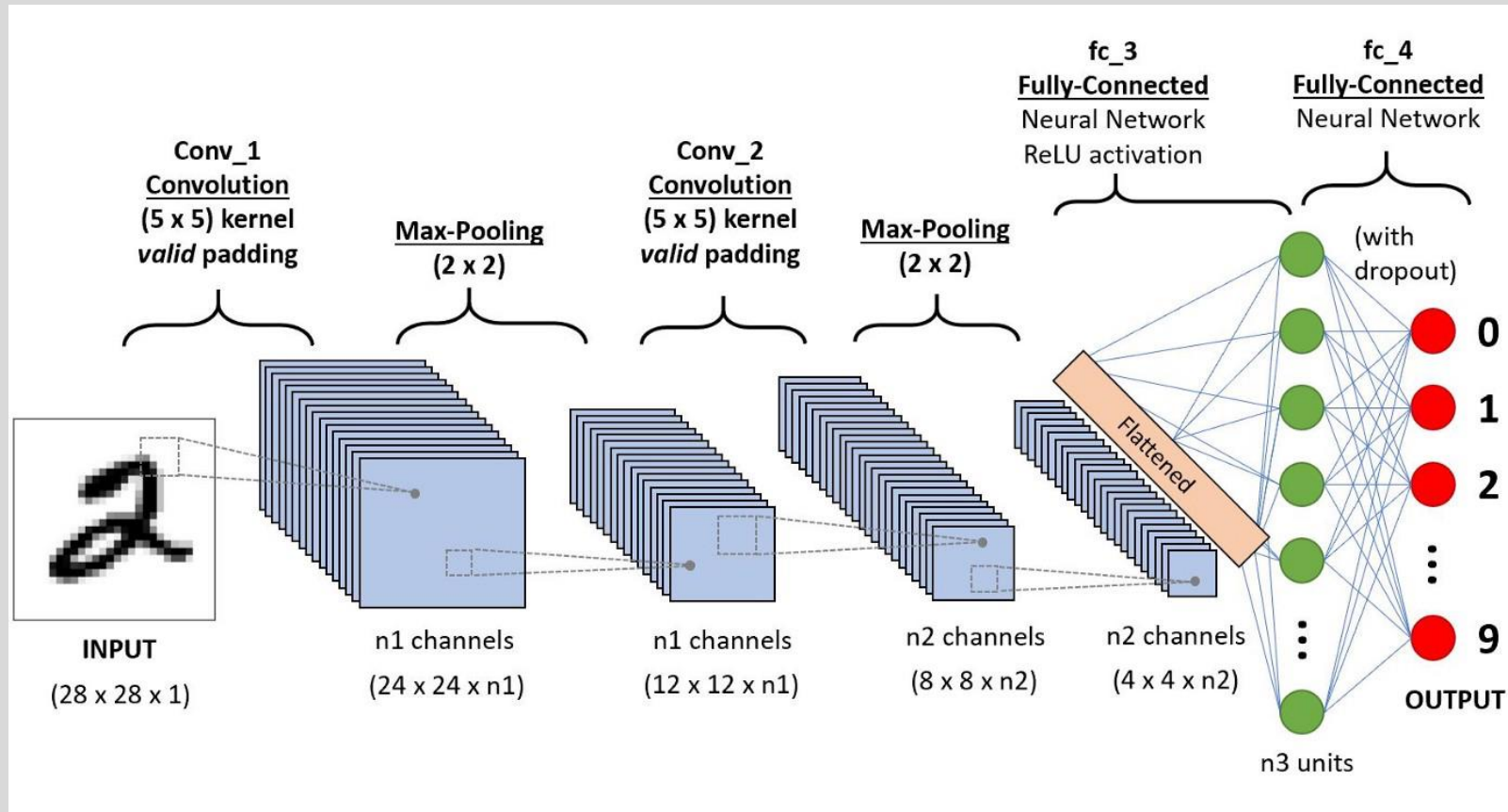
Identically distributed

- Standard machine learning assumes samples with the same features have the same label probability distribution
- Nodes in a graph may have the same features but different label probability distributions when conditioned on their neighbors



Modern ML toolbox assumes regularity

Standard ML methods and deep learning assume fixed-sized and ordered inputs



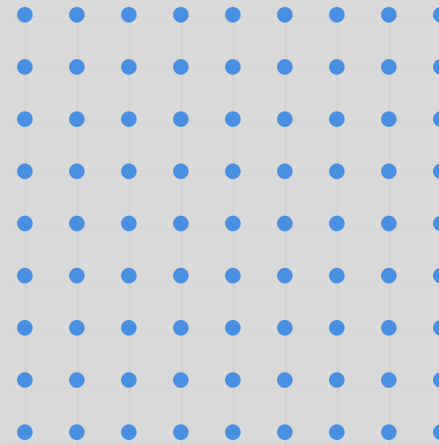


Modern ML toolbox assumes regularity

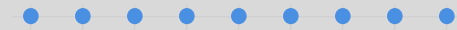
- Graphs contain arbitrary node and edge counts
- Graph nodes do not have an objective ordering
- Different node types may have different features



Graph Data



Images

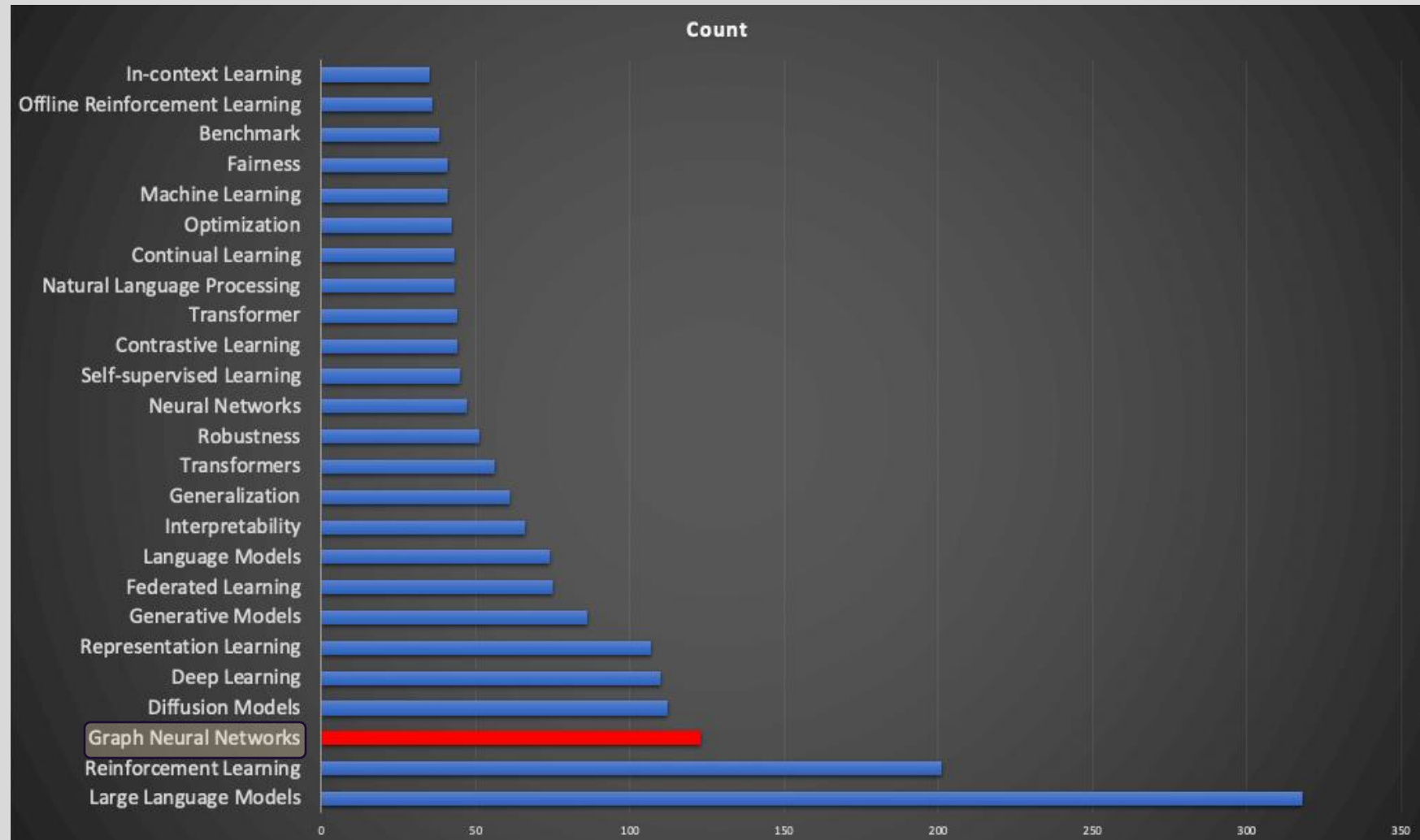


Text



Graph learning is a hot topic

Top 25 keywords ICLR 2024





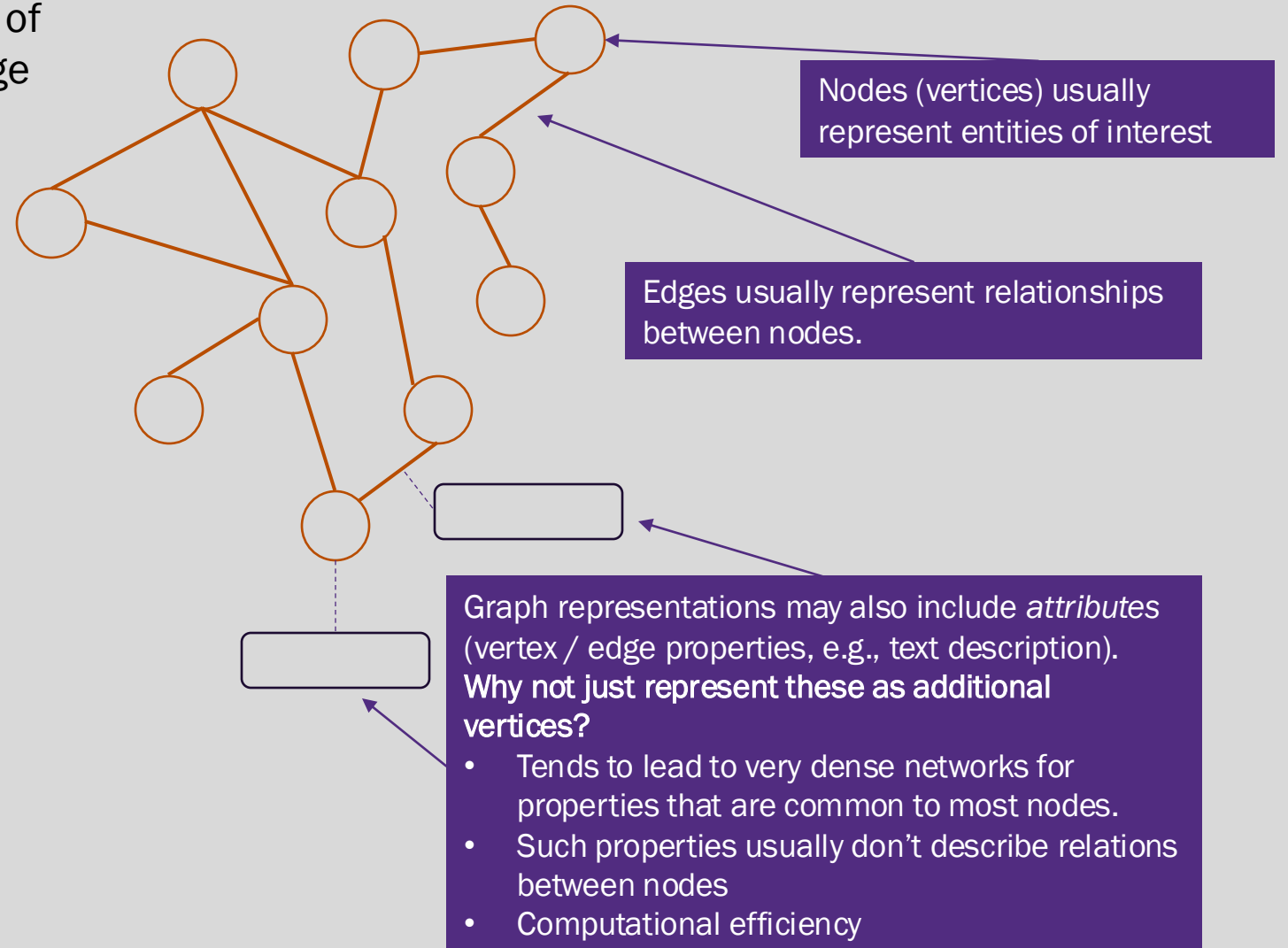
Terminology





What is a graph?

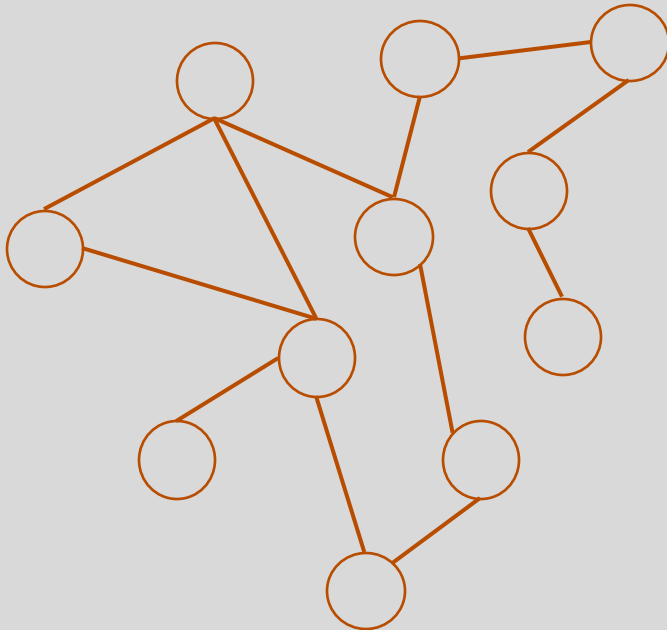
A graph, $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, is defined by a set of nodes, $\mathcal{V}$, and a set of edges, $\mathcal{E}$. An edge from node $u \in \mathcal{V}$ to node $v \in \mathcal{V}$ is denoted as $(u, v) \in \mathcal{E}$.



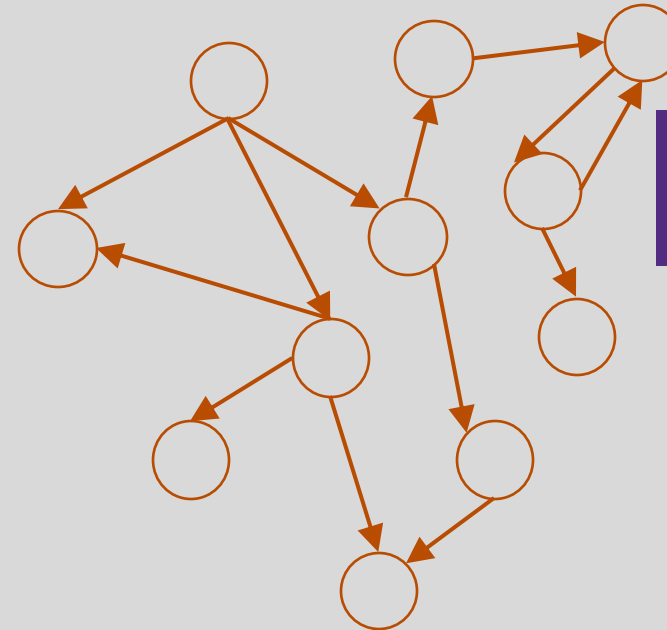


Graph directionality

Undirected Graph – edges have no direction and define relations that are inherently bidirectional.



Directed Graph – edges have a specified direction and define relations that are inherently unidirectional.

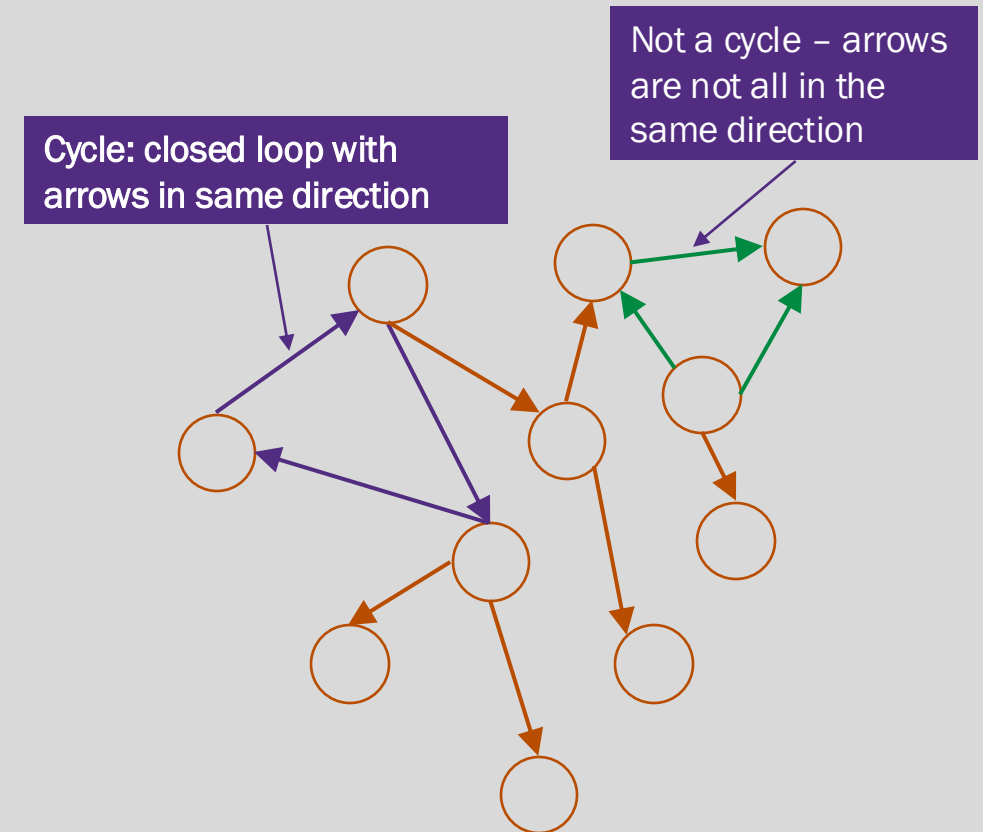


Directed graphs may contain edges in both directions between nodes.



Acyclic graphs

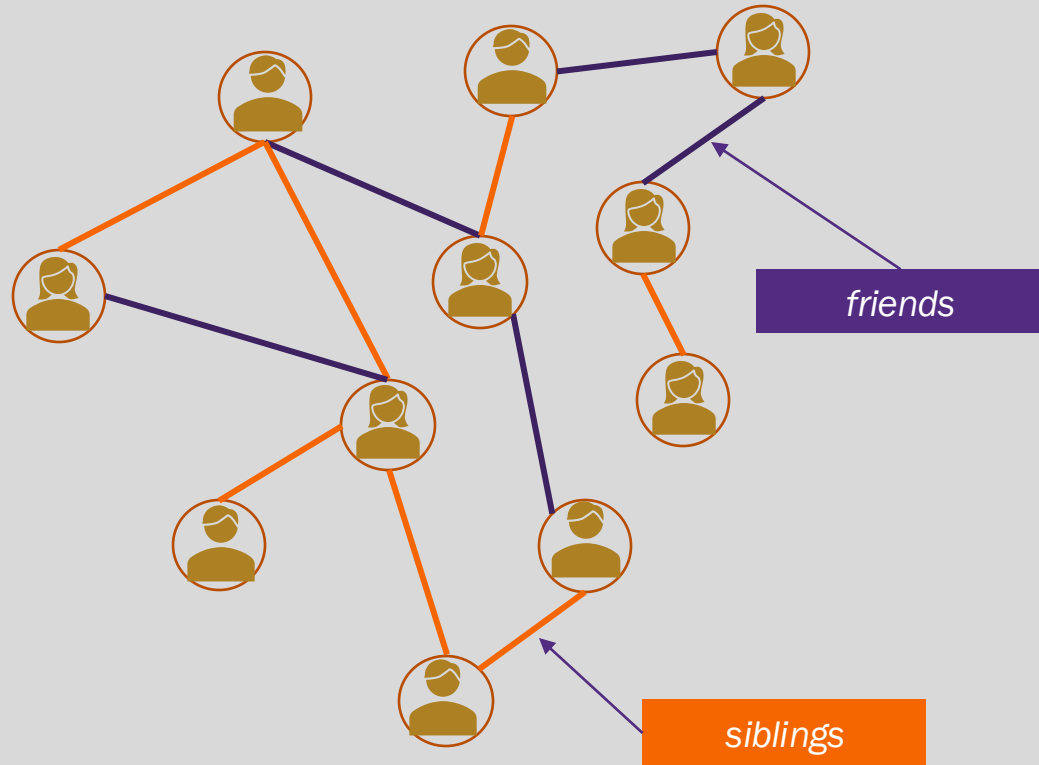
- A cycle is a closed loop of edges in a directed graph where the edges point in the same direction
- Directed acyclic graph (DAG) - a directed graph without any cycles
- Example: citation network where papers can only cite previously published papers





Multi-relational graphs

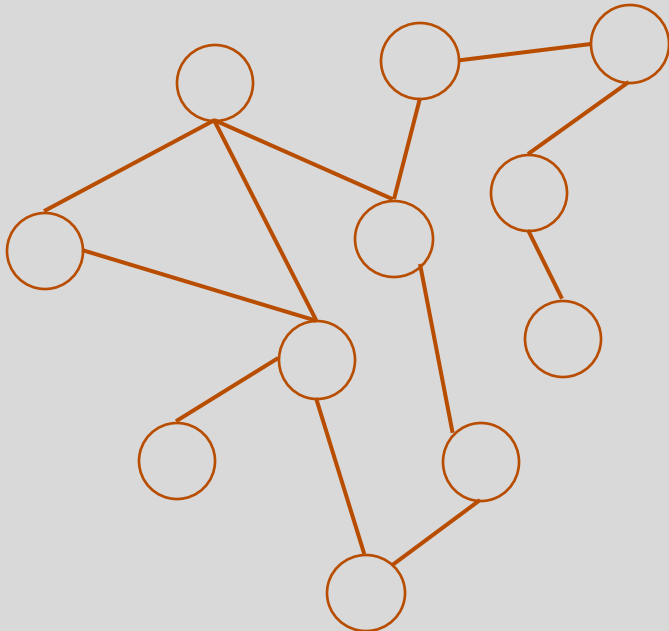
A *multi-relational* graph has more than one relationship type. An edge from node $u \in \mathcal{V}$ to node $v \in \mathcal{V}$ with the relationship type, τ , is denoted as $(u, \tau, v) \in \mathcal{E}$.



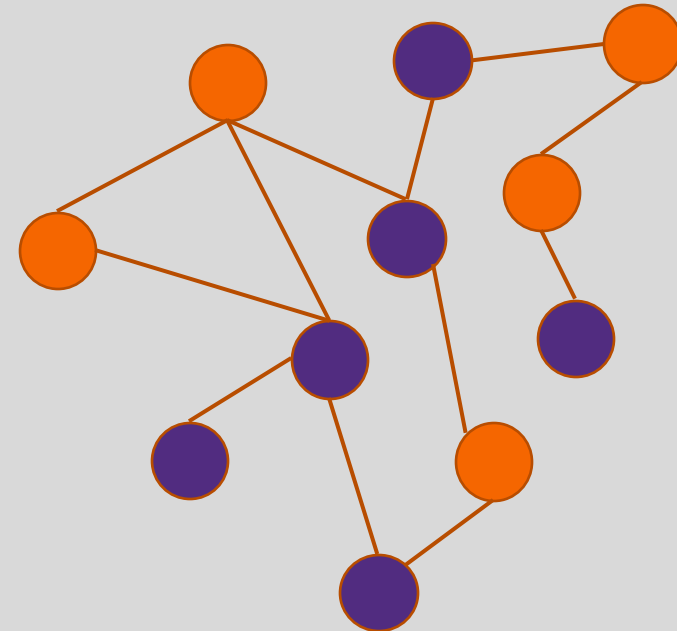


Heterogeneous graphs

Homogeneous graph– all nodes are of the same type and have the same attributes



Heterogeneous graph– different node types with possibly different attributes. Such nodes can be partitioned into disjoint sets.



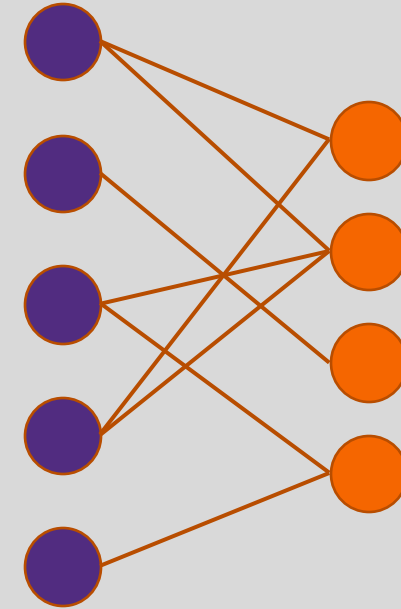


Bipartite graph

A graph in which the nodes, $\mathcal{V}$, can be divided into two $\mathcal{V}_1$ and $\mathcal{V}_2$ such that every edge connects a node in $\mathcal{V}_1$ to a node in $\mathcal{V}_2$

Examples:

- Recommender networks
- Keyword indices
- Attribution networks (authors-papers, actors-movies)
- Drugs to diseases treated
- Restricted Boltzmann machines

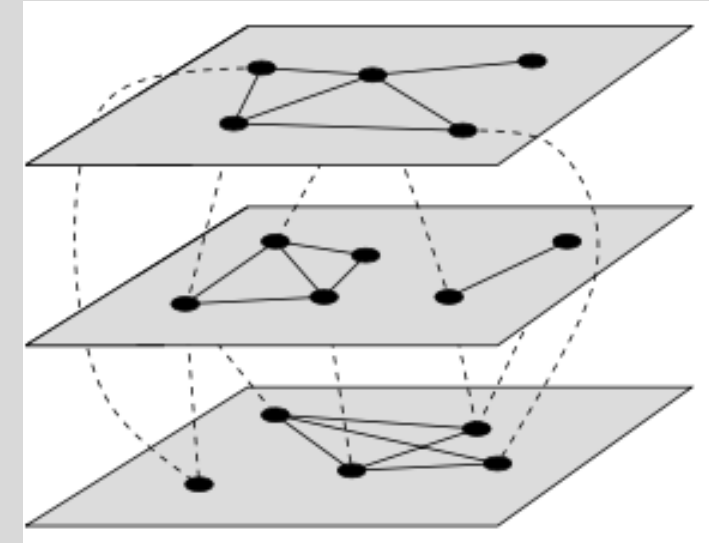


All edges in a **bipartite graph** are between nodes of two distinct types.

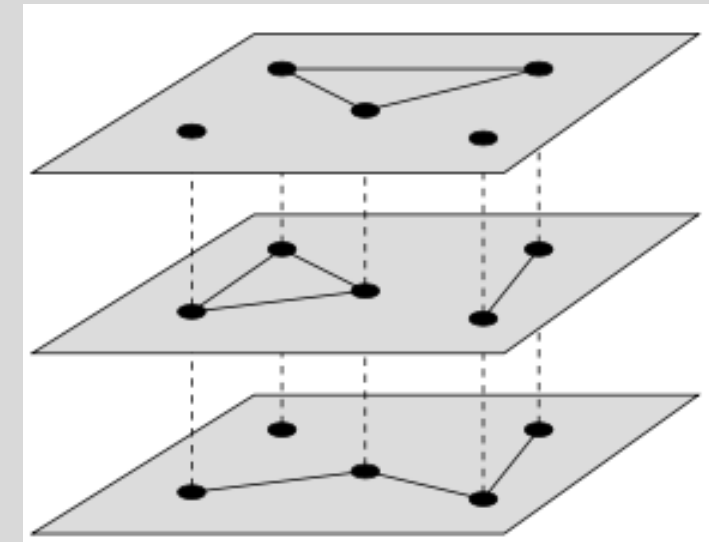
Multilayer and multiplex networks

- Multilayer networks contain a set of layers
 - Each layer is a distinct network with different node types and connectivity
 - Interlayer edges connect nodes between layers
 - Example: transportation network where layers represent airlines, trains, and buses where interlayer connections indicate shared location.
- Multiplex networks
 - Special case of multilayer network where the nodes are the same but connectivity changes
 - Example: social network with different relationship types at each layer
- Dynamic networks
 - Nodes & connections change over time → multilayer
 - Only connections change over time → multiplex

Multilayer network



Multiplex network





Trees

- A tree is a connected, undirected network with no loops
- Connected means that every node is reachable from every other node (in a tree, there is exactly one path)
- Undirected and with no loops is distinct from a directed acyclic graph where the definition of a loop requires that all edges in the loop are in the same direction
 - A DAG with edges removed may not be a tree
- A tree with n nodes will have exactly $n - 1$ edges

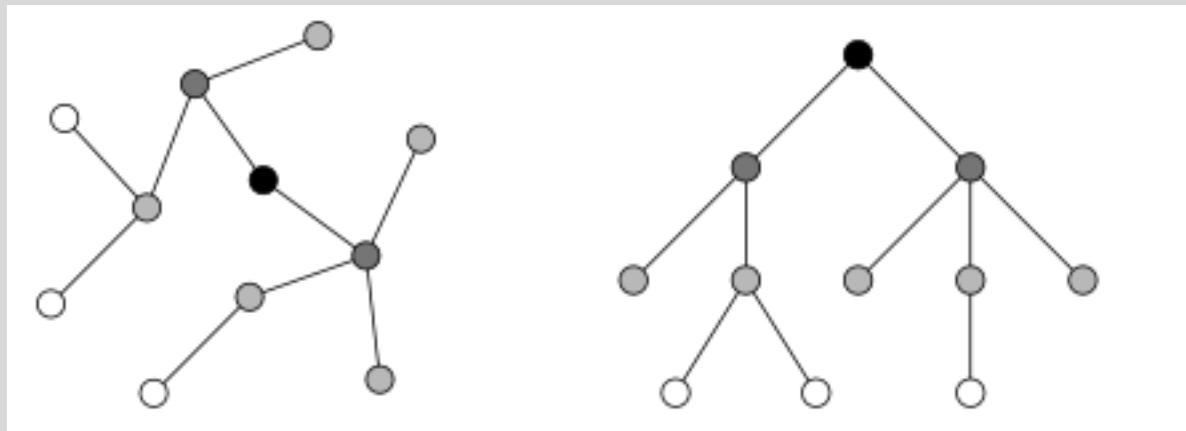


Image credit: Newman 2018

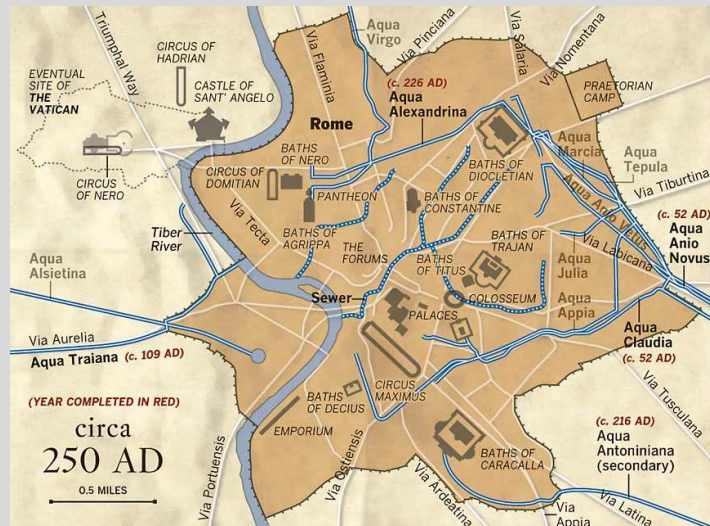


Graph or network?

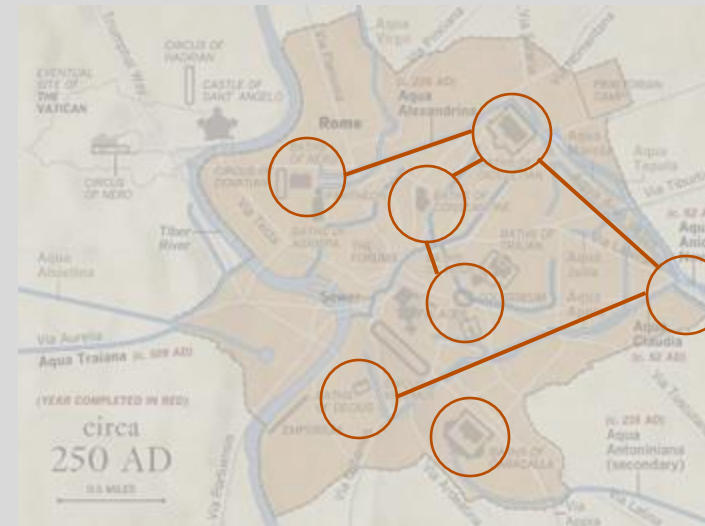
Network – the real-world system or phenomena of interest

Graph – a model of the network

Network



Graph (model)





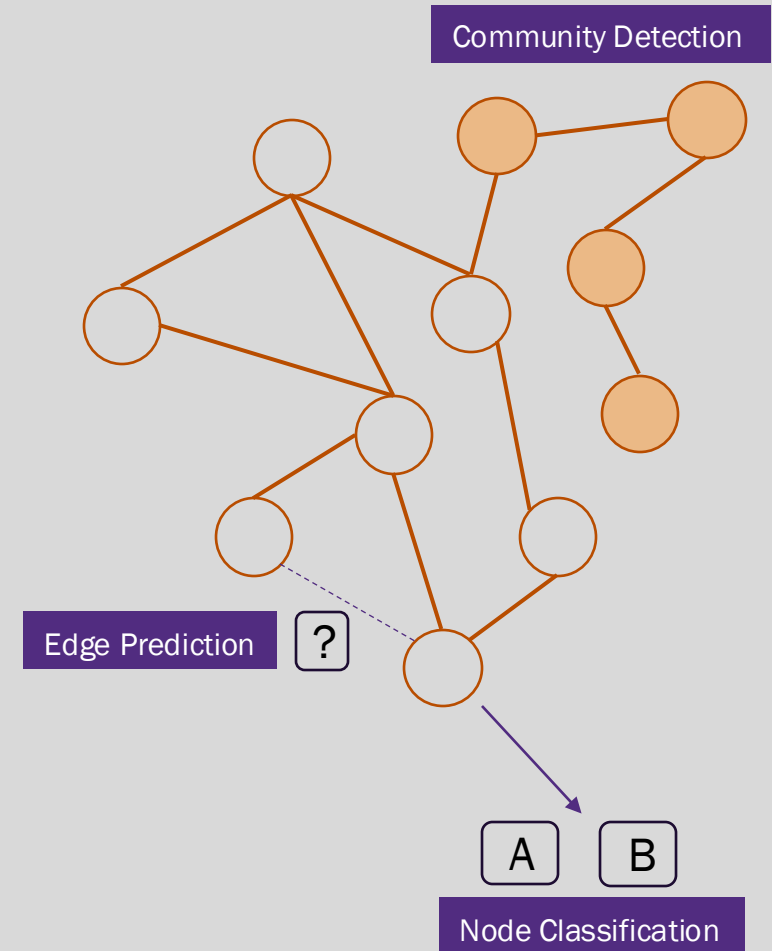
Machine learning tasks on graphs





Intra-graph machine learning tasks

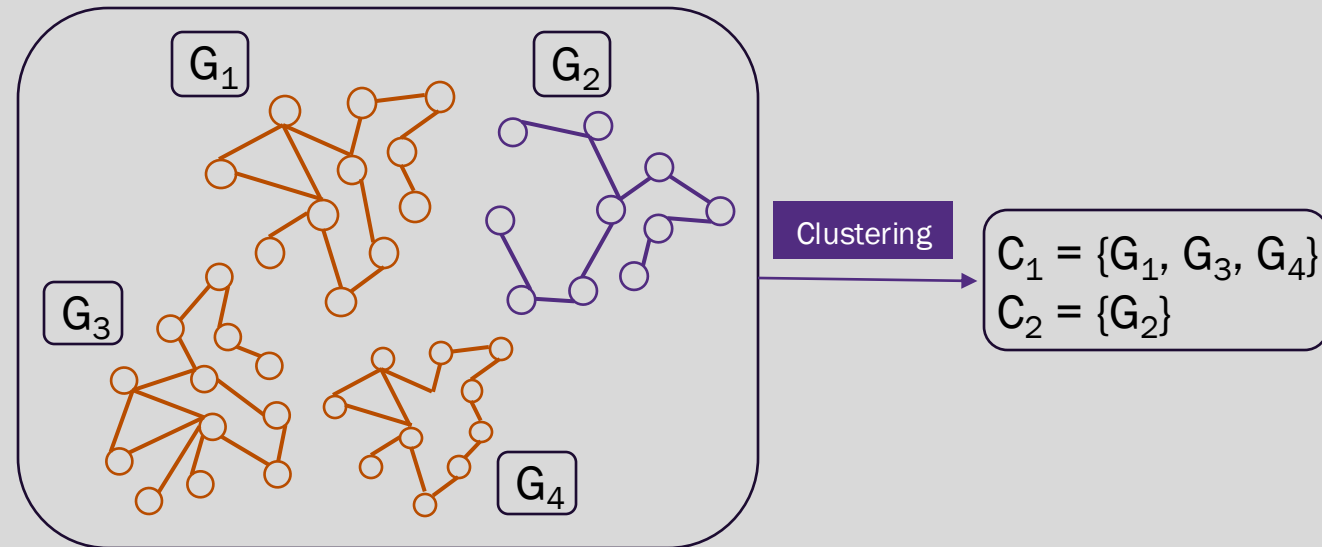
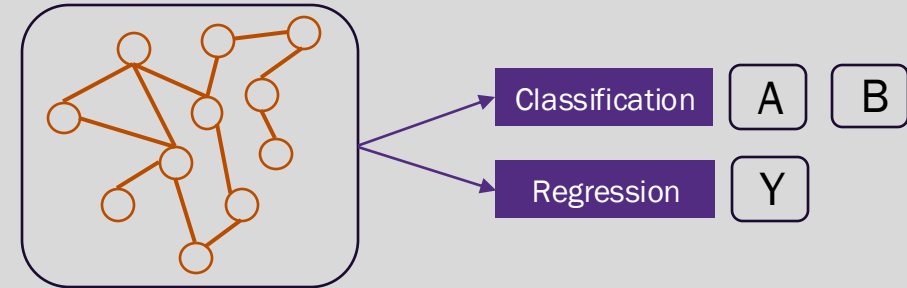
- Node classification – given information about the node (may include neighbors, relationships, attributes), infer class membership (usually an attribute)
- Edge prediction – given information about two nodes, determine if a given relation exists between them
- Community detection (node clustering) – detection of “meaningful” node groups





Whole graph and inter-graph machine learning tasks

- Whole graph classification and regression – given one entire graph, infer class membership or estimate (regress) a related numerical property
- Inter graph tasks – given multiple graphs estimate the similarity between graphs and/or identify clusters





AlphaFold

- Proteins represented as graphs
 - Nodes: amino acids of protein
 - Edges: proximity between amino acids
- Goal: predict the 3D spatial coordinates of each amino acid (node) from input protein sequence

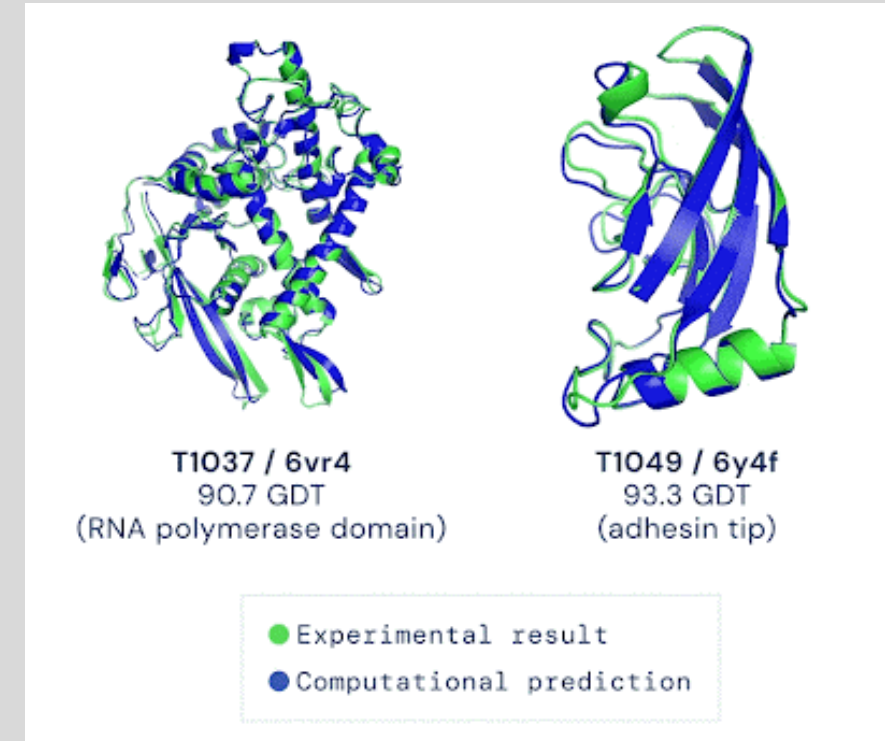


Image credit: [Deepmind](#)



PinSage: Graph based recommendation

- Model Pinterest as a bipartite graph of “pins” and “board” nodes. Pins are associated with text and image features.
 - Goal is to recommend “pins” (on Pinterest)
 - View recommendations as *link predictions* between pins and boards
- Graph model used to form embeddings of pins that serve as “queries” for pin recommendation



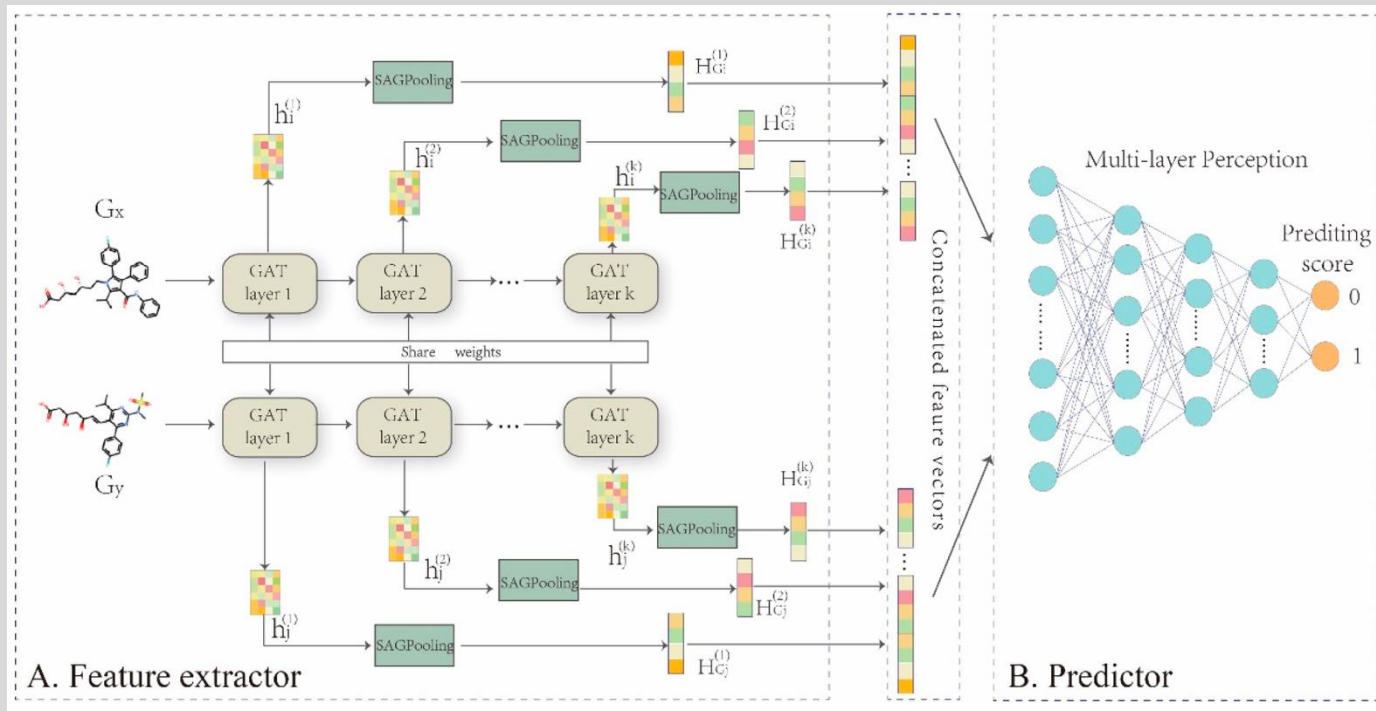
Poor recommendation

Good recommendation



Ying, Rex, et al. "Graph convolutional neural networks for web-scale recommender systems."
Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining. 2018.

Drug-drug interaction prediction



- Each drug is represented by a graph
- Each drug in a candidate DDI pair is passed through a 5-layer GCN with attention
- Node representations are down-sampled to form a representation of each drug
- Graph classification: drug representations are concatenated and passed through a FFN for DDI classification